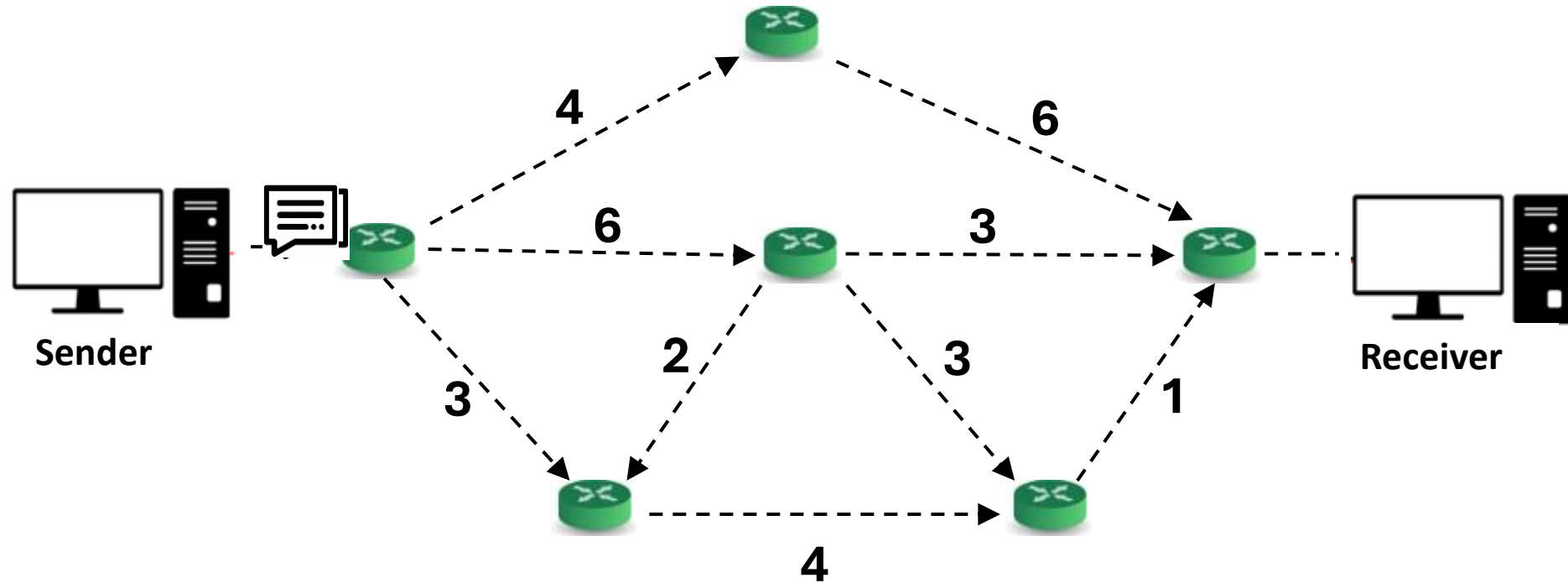
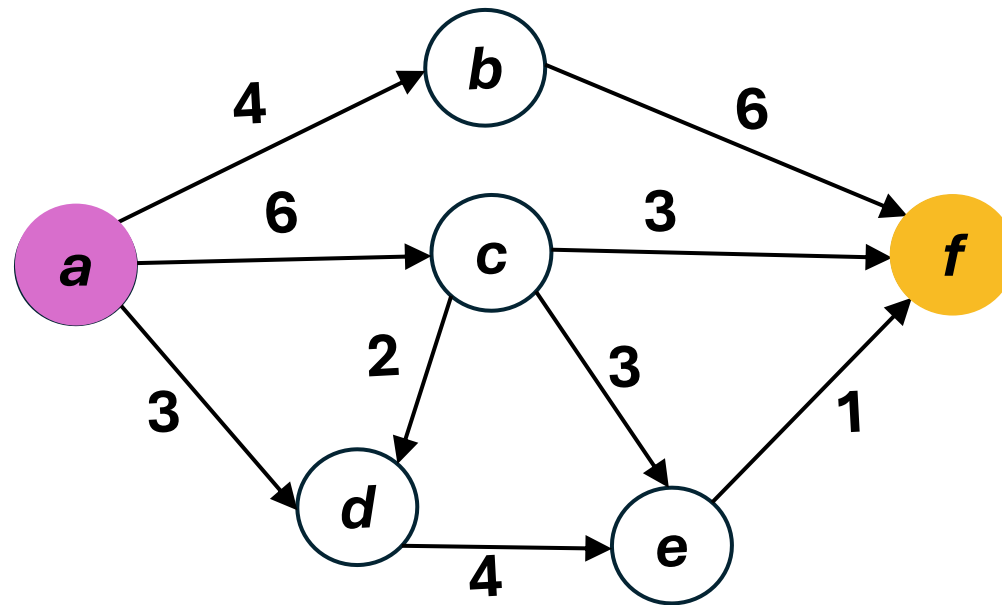


Dijkstra's algorithm in practice: Shortest network delay (adapted from LeetCode #743)

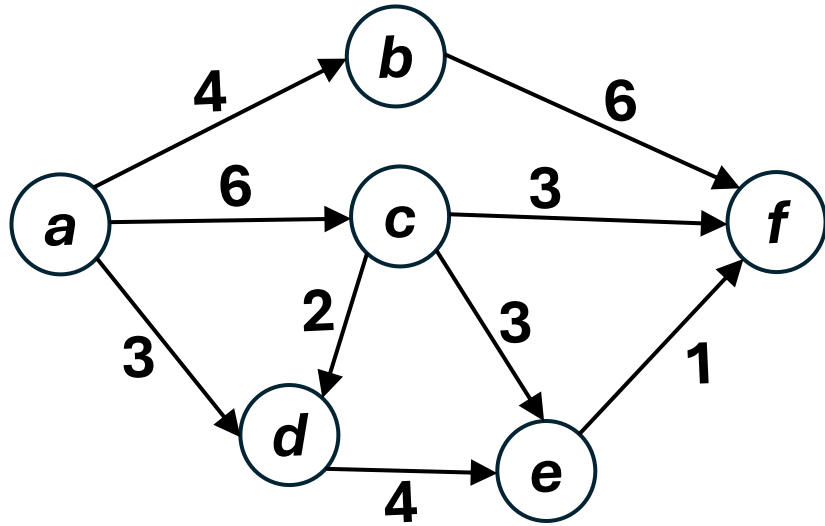


Dijkstra's algorithm in practice: Shortest network delay (adapted from LeetCode #743)

Model the network as a graph:



Preparations

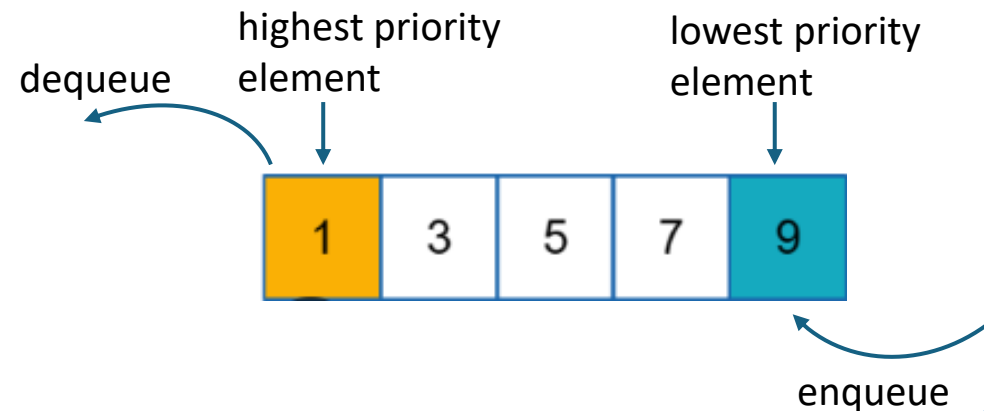
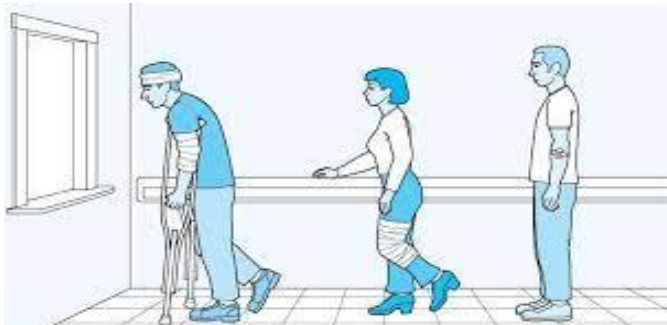


frontier
(priority queue)

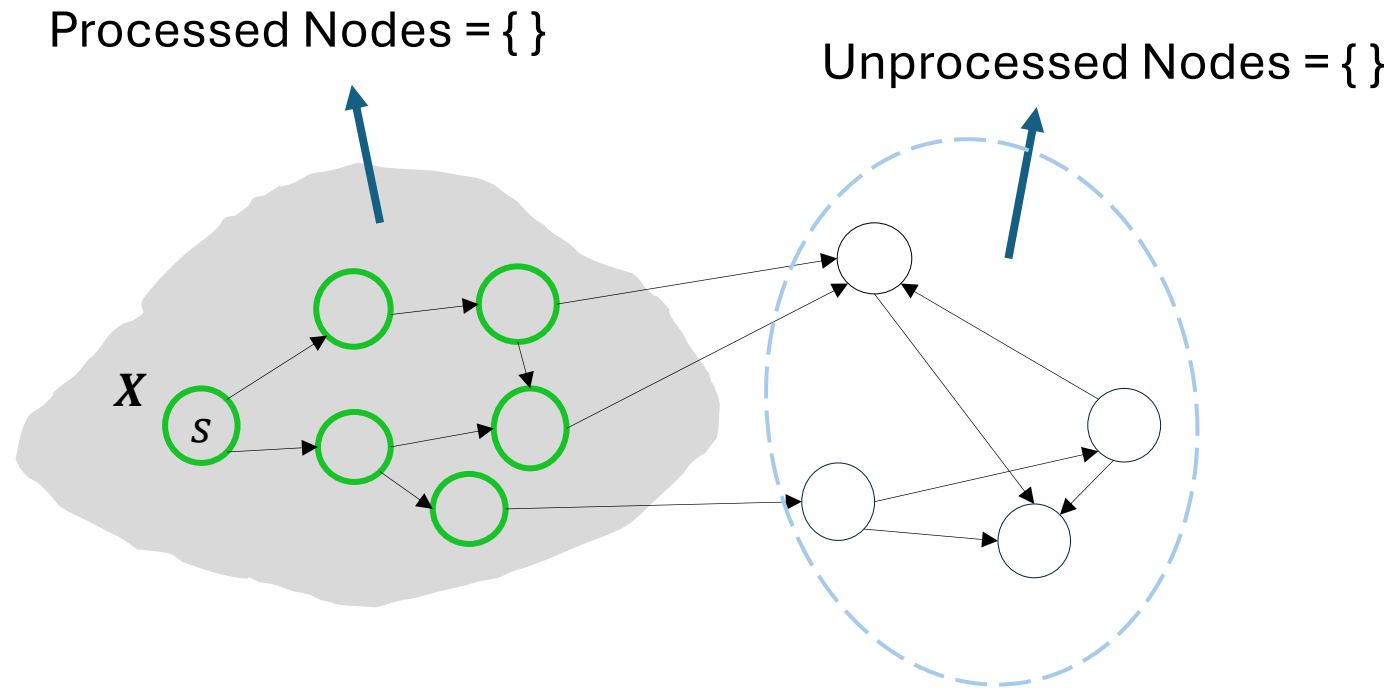
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>		
<i>b</i>		
<i>c</i>		
<i>d</i>		
<i>e</i>		
<i>f</i>		

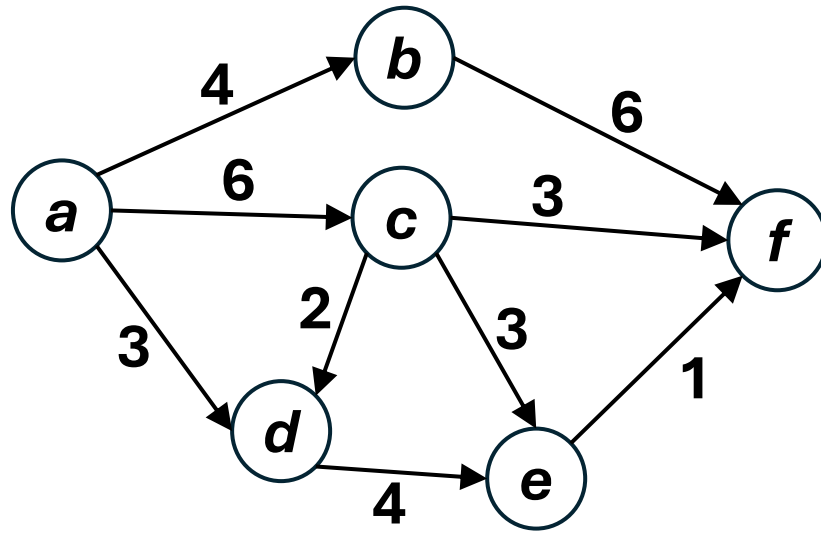
Priority queue:



Preparations



Initialisations



frontier
(priority queue)

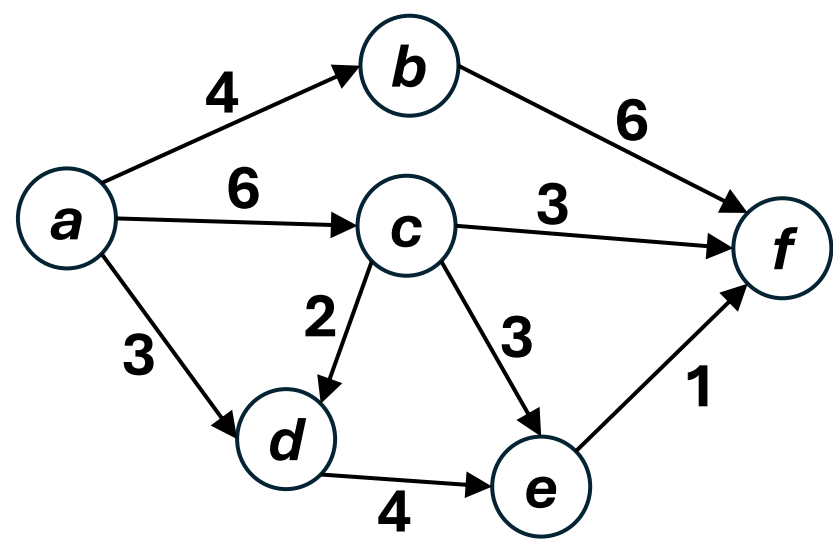
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	∞	--
<i>b</i>	∞	--
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = { }

Initialisations



frontier
(priority queue)

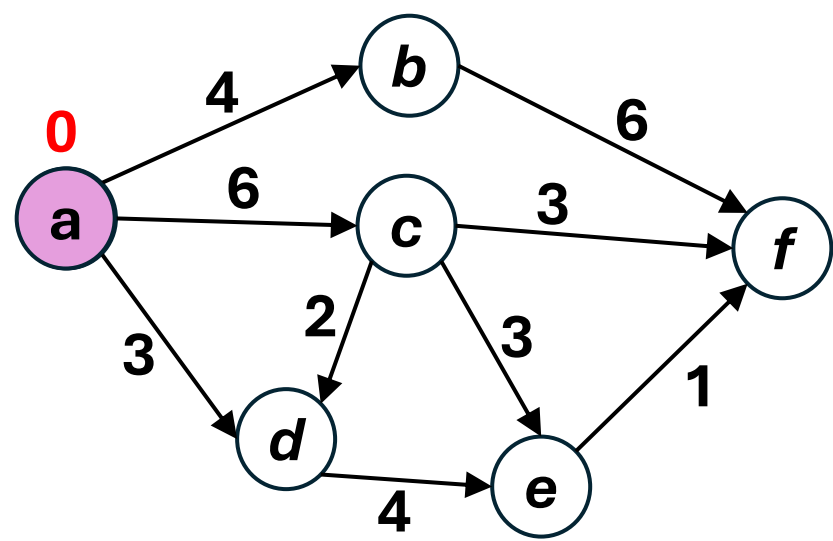
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	∞	--
<i>b</i>	∞	--
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Initialisations (continued)



frontier
(priority queue)

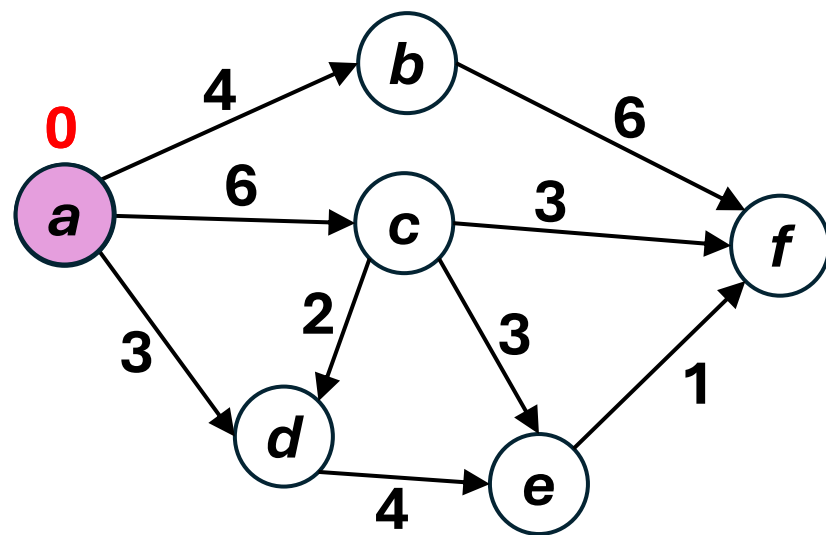
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	∞	--
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Initialisations (continued)



frontier
(priority queue)

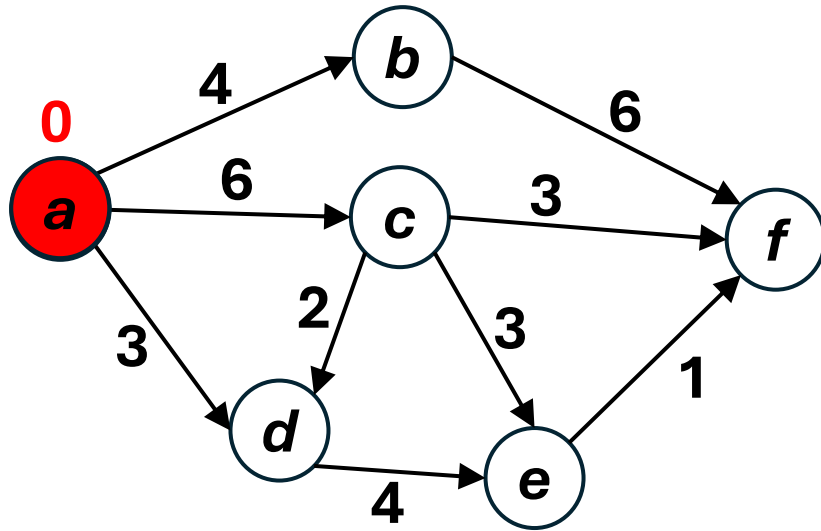
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	∞	--
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Main loop: Iteration 1



frontier
(priority queue)

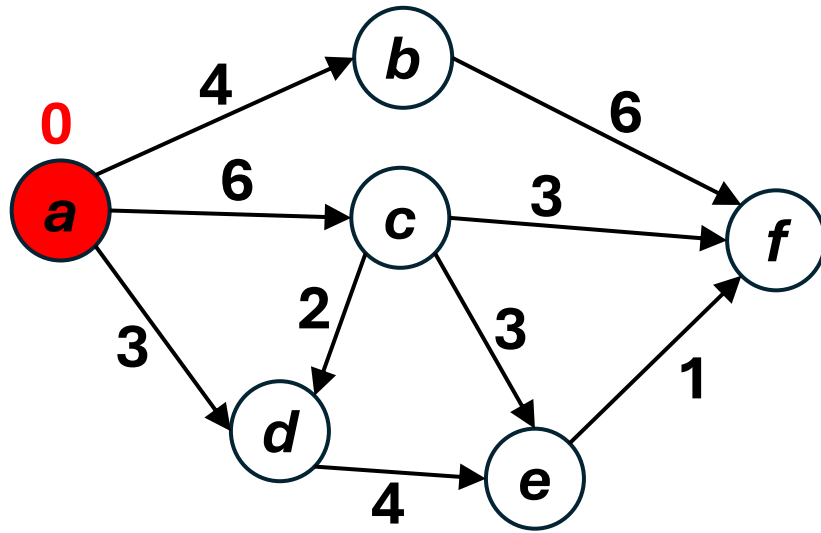
node	dist
	0

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	∞	--
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1



frontier
(priority queue)

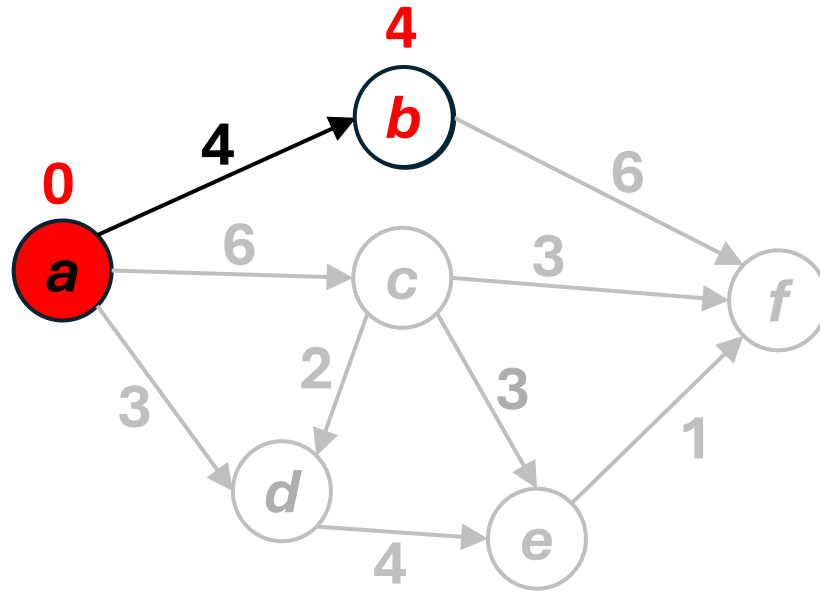
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	∞	--
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1-1



$$\textit{dist}[b] = 4$$

frontier (priority queue)

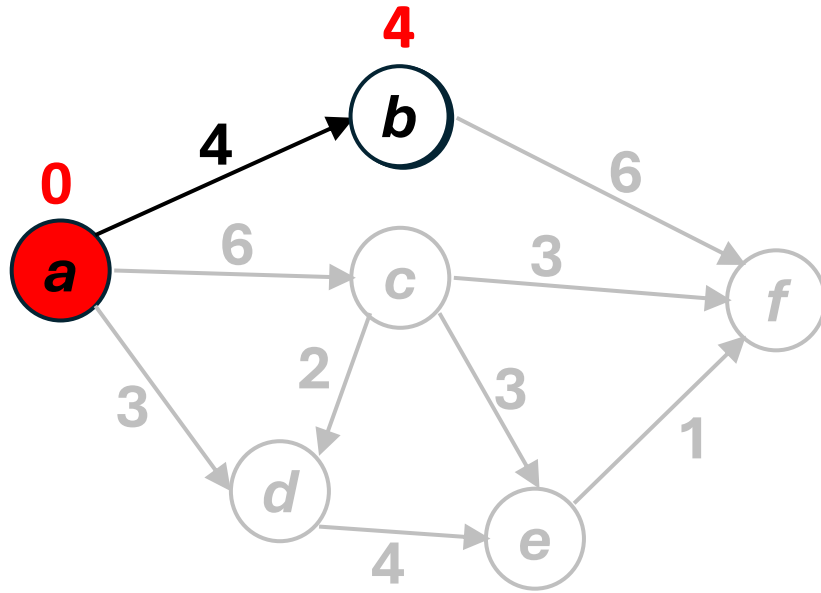
node	dist
------	------

node	shortest distance from a	previous node
a	0	--
b	4	a
c	∞	--
d	∞	--
e	∞	--
f	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f}

Iteration 1-1



frontier
(priority queue)

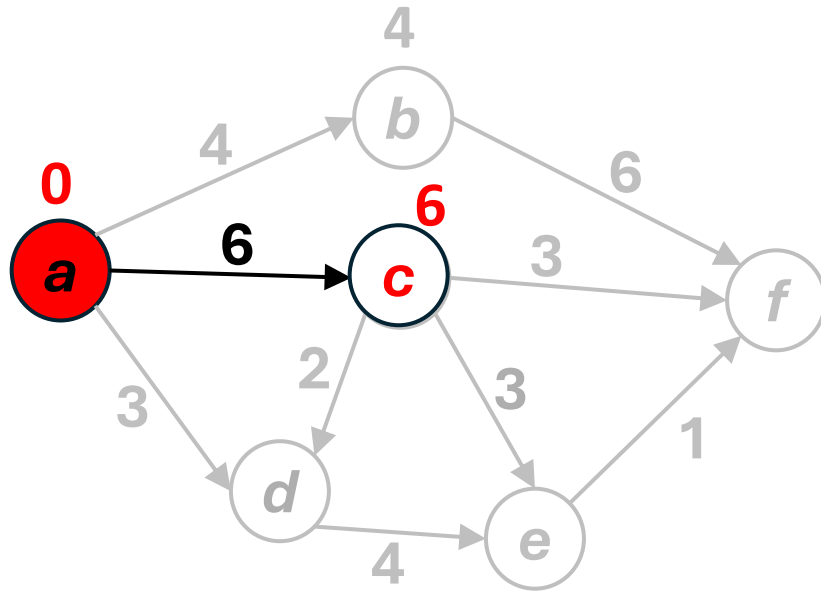
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	∞	--
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1-2



$dist[c] = 6$

frontier
(priority queue)

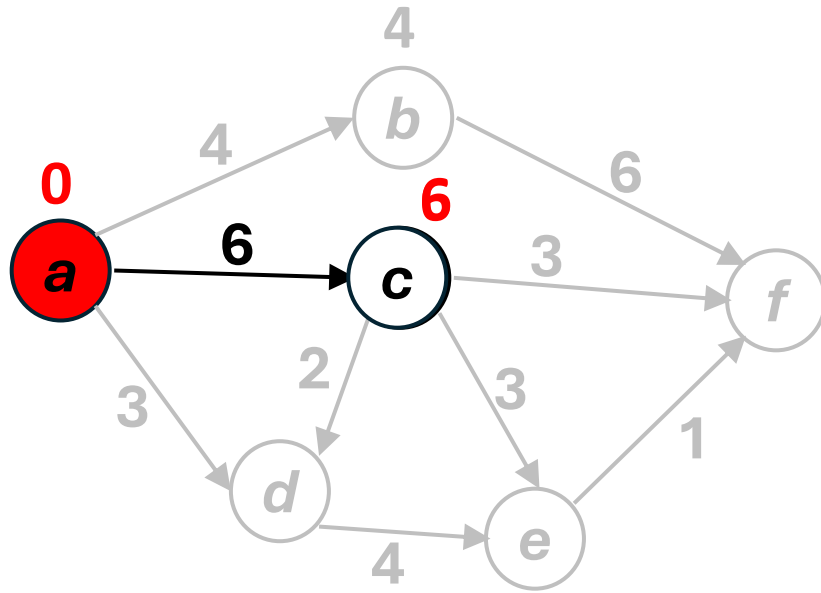
node	dist
b	4

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1-2



frontier
(priority queue)

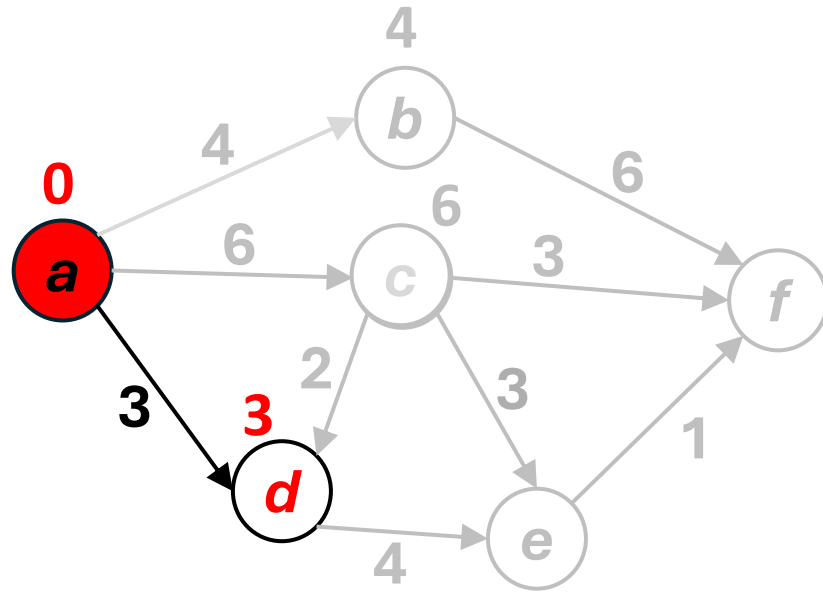
node	dist
b	4

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	∞	--
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1-3



$dist[d] = 3$

frontier
(priority queue)

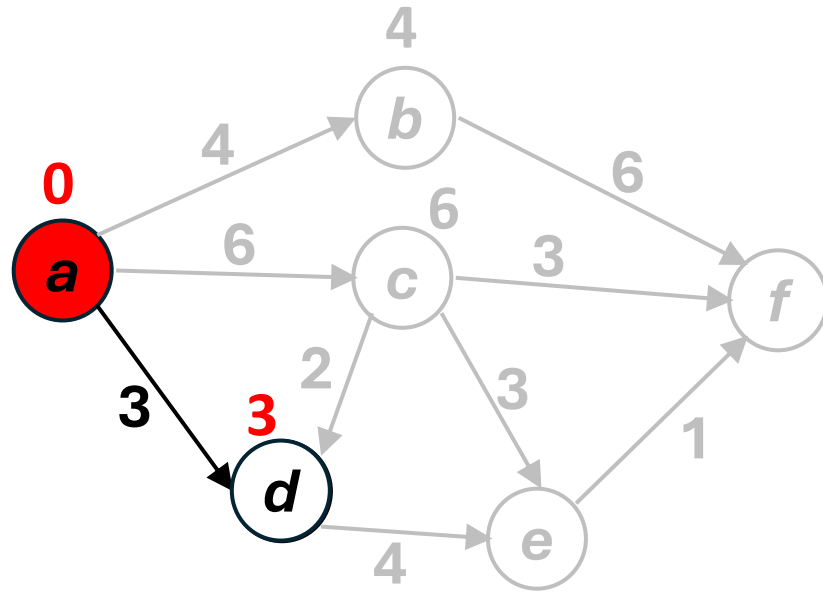
node	dist
b	4
c	6

Processed Nodes = { }

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
d	3	a
<i>e</i>	∞	--
<i>f</i>	∞	--

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1-3



frontier
(priority queue)

Ordered by
priority

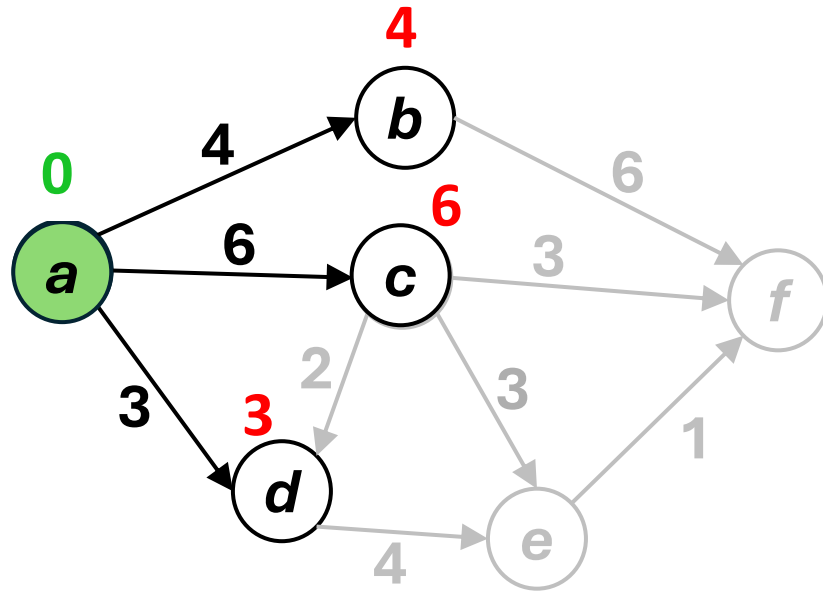
node	dist
b	4
c	6

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = { }

Unprocessed Nodes = {a, b, c, d, e, f }

Iteration 1



frontier
(priority queue)

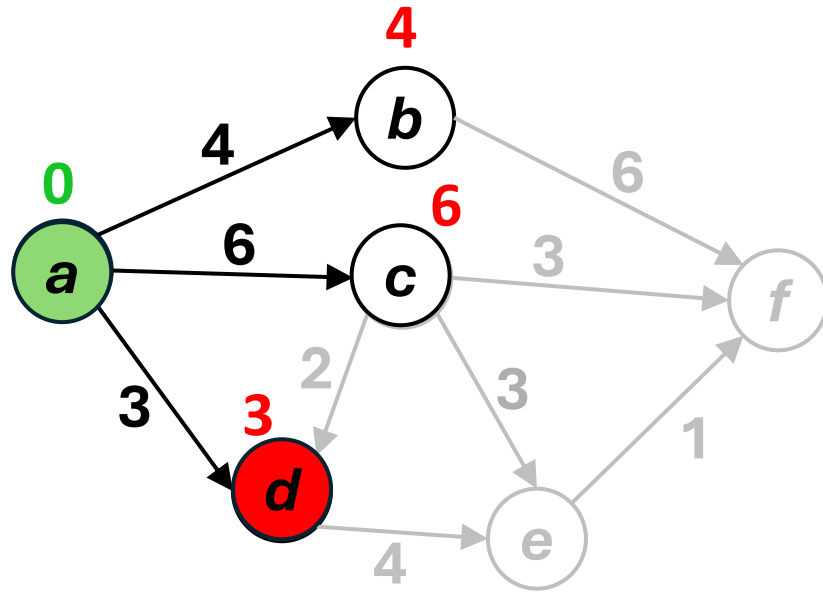
node	dist
<i>d</i>	3
<i>b</i>	4
<i>c</i>	6

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	∞	--
<i>f</i>	∞	--

Processed Nodes = {*a*}

Unprocessed Nodes = {*b*, *c*, *d*, *e*, *f*}

Iteration 2



frontier
(priority queue)

node	dist
<i>d</i>	3
<i>b</i>	4
<i>c</i>	6

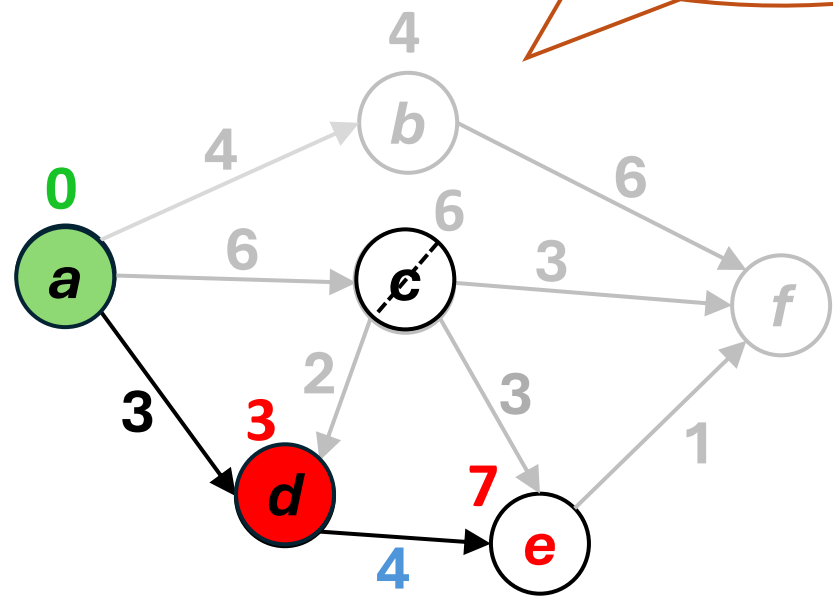
Processed Nodes = {*a*}

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	∞	--
<i>f</i>	∞	--

Unprocessed Nodes = {*b*, *c*, *d*, *e*, *f*}

Iteration 2

We only consider outgoing neighbours for directed graphs.



frontier
(priority queue)

node	dist
b	4
c	6

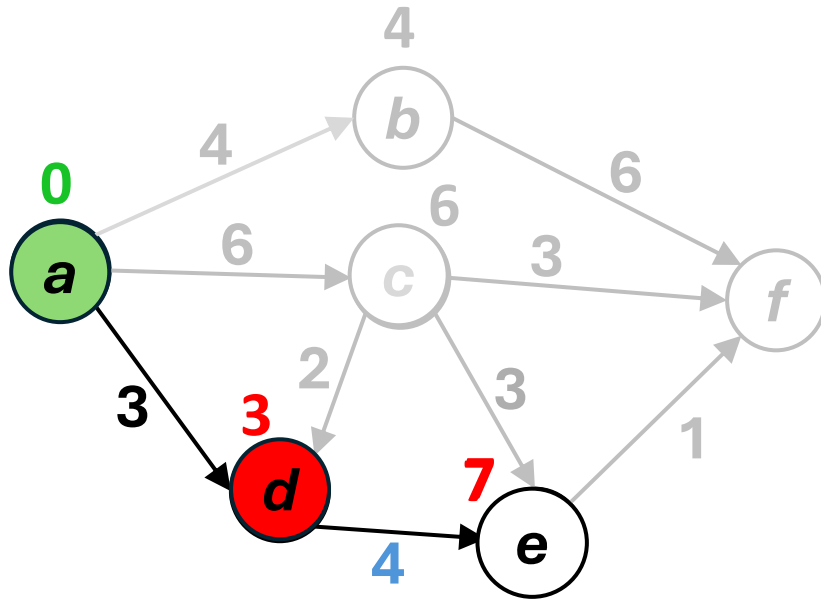
$$\begin{aligned}
 \text{dist}[e] &= \text{dist}[d] + 4 \\
 &= 3 + 4 \\
 &= 7
 \end{aligned}$$

Processed Nodes = {a}

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	∞	--

Unprocessed Nodes = {b, c, d, e, f }

Iteration 2



frontier
(priority queue)

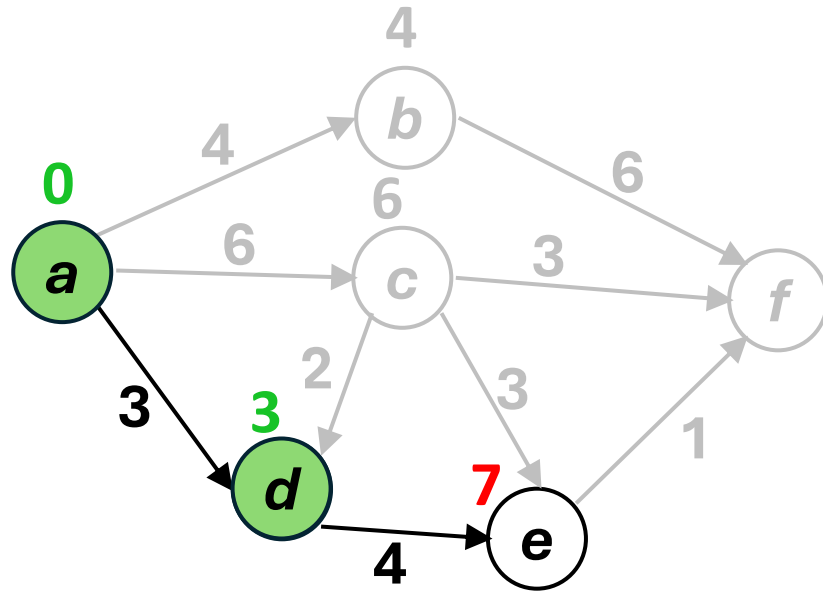
node	dist
<i>b</i>	4
<i>c</i>	6

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	∞	--

Processed Nodes = {*a*}

Unprocessed Nodes = {*b*, *c*, *d*, *e*, *f*}

Iteration 2



frontier
(priority queue)

node	dist
b	4
c	6
e	7

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	∞	--

Processed Nodes = {**a**, **d**}

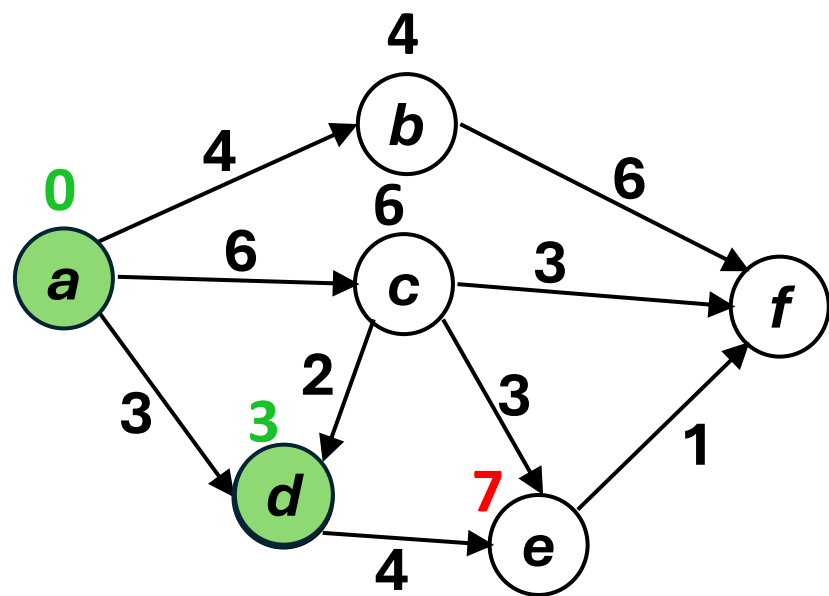
Unprocessed Nodes = {**b**, **c**, **e**, **f**}

Quick question



Which of the following nodes will be processed next?

- (a) *b* ✓
- (b) *c*
- (c) *e*
- (d) *f*

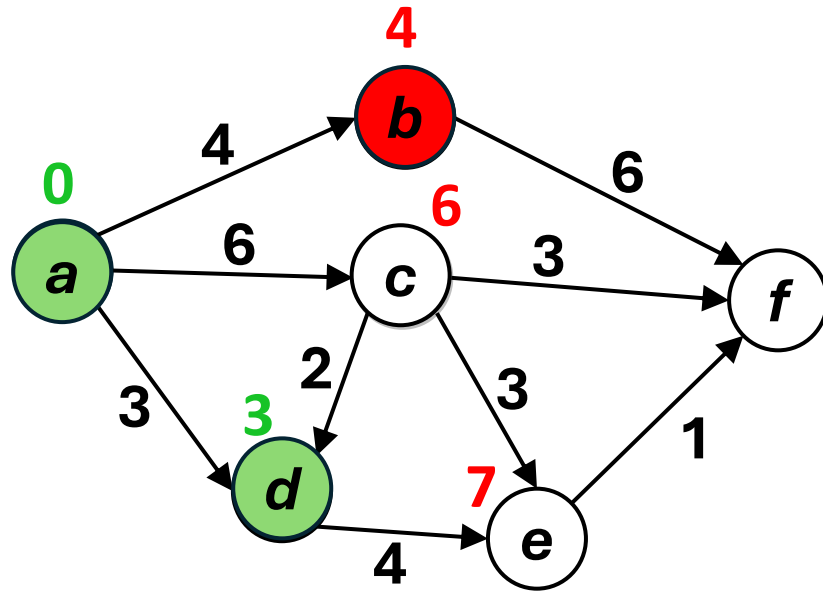


frontier
(priority queue)

node	dist
b	4
c	6
e	7

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	∞	--

Iteration 3



frontier
(priority queue)

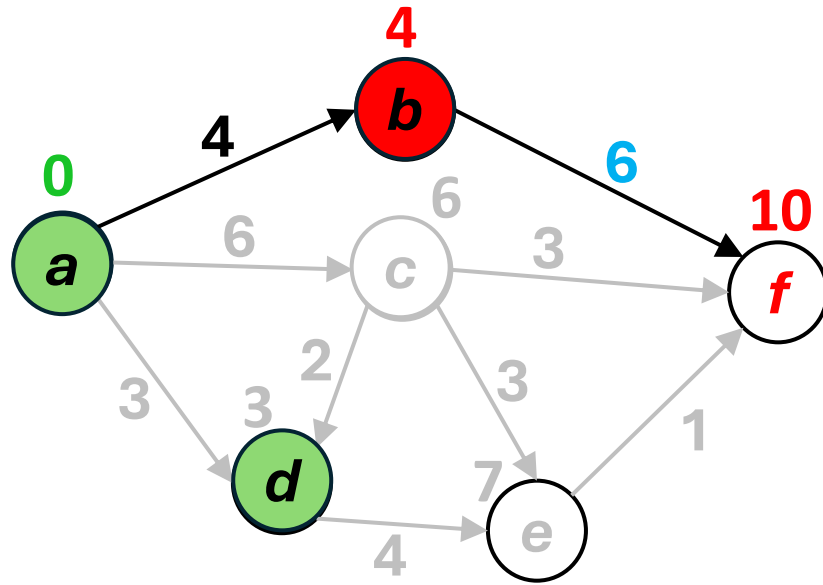
node	dist
b	4
c	6
e	7

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	∞	--

Processed Nodes = {**a**, **d**}

Unprocessed Nodes = {**b**, **c**, **e**, **f**}

Iteration 3



$$\begin{aligned} \text{dist}[f] &= \text{dist}[b] + 6 \\ &= 4 + 6 \\ &= 10 \end{aligned}$$

Processed Nodes = {a, d}

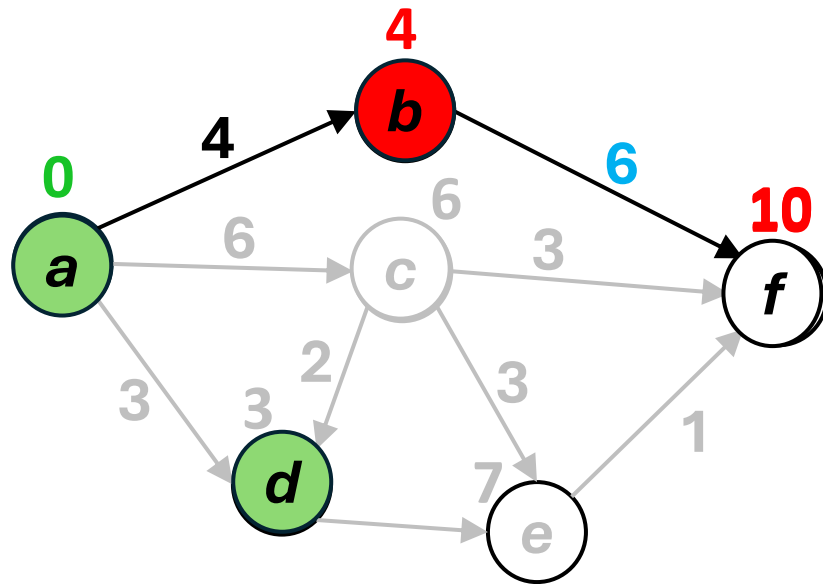
frontier
(priority queue)

node	dist
c	6
e	7

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	10	<i>b</i>

Unprocessed Nodes = {b, c, e, f}

Iteration 3



frontier
(priority queue)

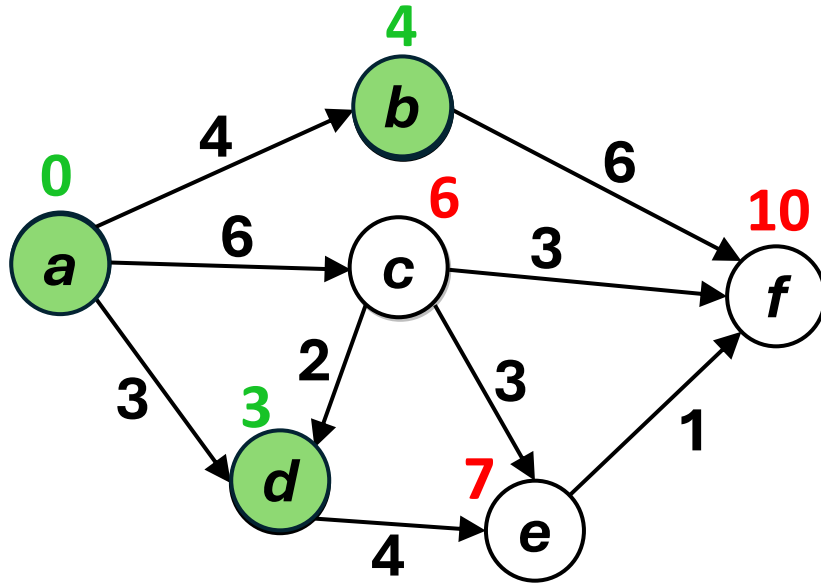
node	dist
c	6
e	7

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	10	<i>b</i>

Processed Nodes = {**a**, **d**}

Unprocessed Nodes = {**b**, **c**, **e**, **f**}

Iteration 3



frontier
(priority queue)

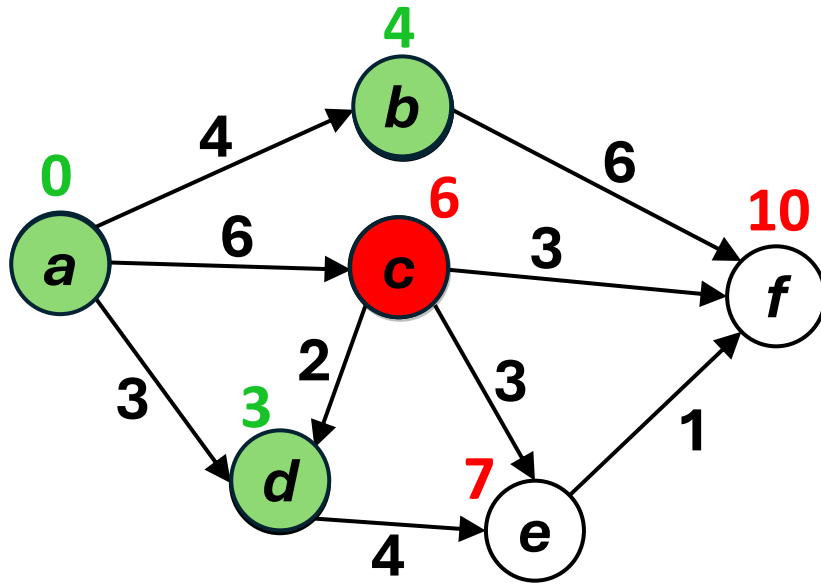
node	dist
c	6
e	7
f	10

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	10	<i>b</i>

Processed Nodes = {**a**, **d**, **b**}

Unprocessed Nodes = {**c**, **e**, **f**}

Iteration 4



frontier
(priority queue)

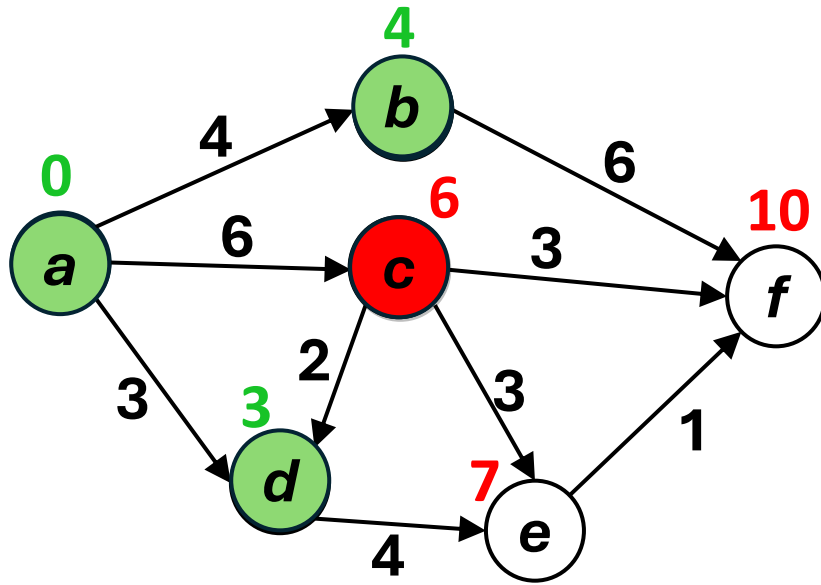
node	dist
c	6
e	7
f	10

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	10	<i>b</i>

Processed Nodes = {**a**, **d**, **b**}

Unprocessed Nodes = {**c**, **e**, **f**}

Iteration 4



frontier
(priority queue)

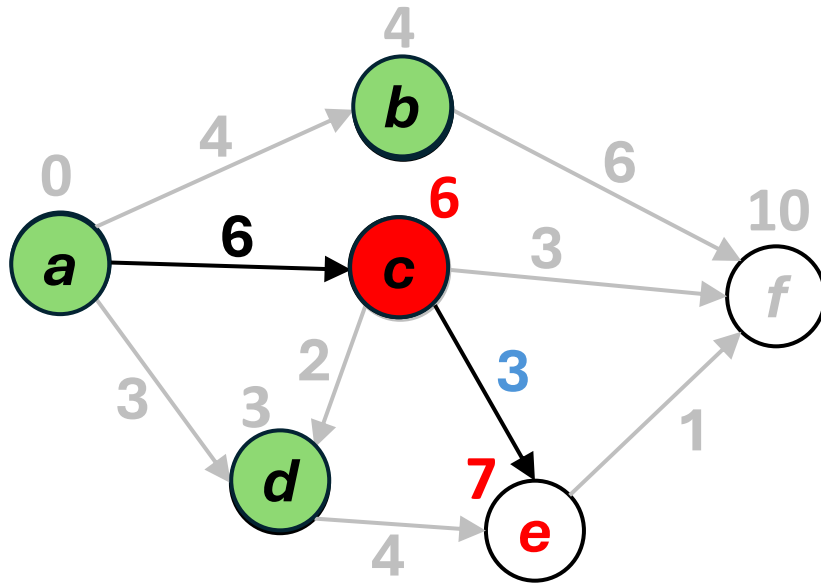
node	dist
e	7
f	10

Processed Nodes = {a, d, b}

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	10	b

Unprocessed Nodes = {c, e, f}

Iteration 4-1



$$\begin{aligned} \text{dist}[e] &= \text{dist}[c] + 3 \\ &= 6 + 3 \\ &= 9 \end{aligned}$$

Processed Nodes = {a, d, b}

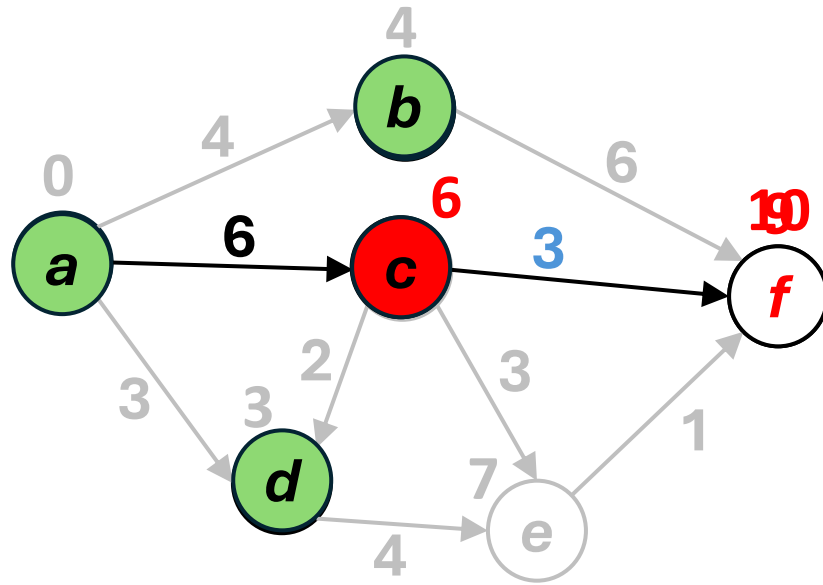
frontier
(priority queue)

node	dist
e	7
f	10

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	10	b

Unprocessed Nodes = {c, e, f}

Iteration 4-2



$$\begin{aligned} \text{dist}[f] &= \text{dist}[c] + 3 \\ &= 6 + 3 \\ &= 9 \end{aligned}$$

Processed Nodes = {a, d, b}

frontier
(priority queue)

node	dist
e	7
f	10

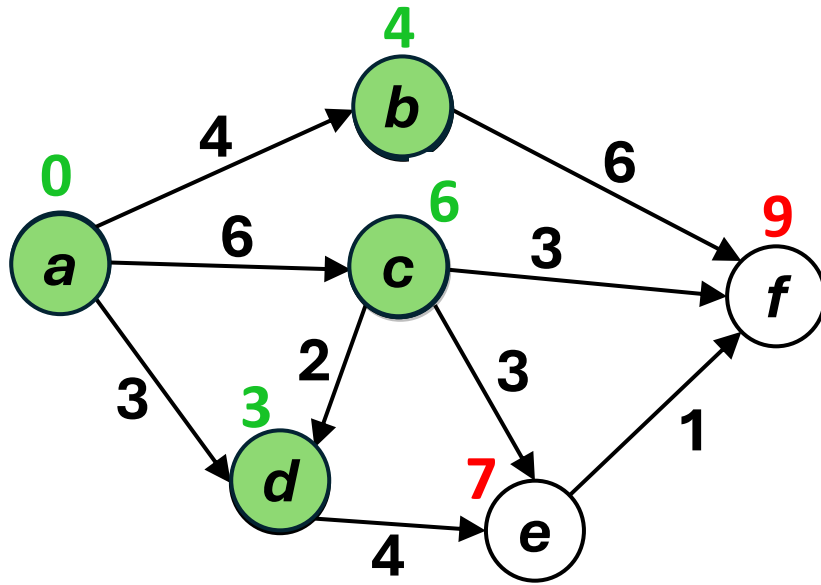
- Start with a **frontier (priority queue)** containing the initial state (source), with path cost = 0.
- Start with an **empty explored set**.
- **Repeat:**
 - If the frontier is empty, then no solution.
 - Remove the frontier node with the **lowest path cost from the source**.
 - If this node is the goal, return the solution.
 - Add the node to the explored set.
 - Expand the node:
 - o For each child, compute its **total path cost from the source**.
 - o If the child is not in the frontier or explored set, **add it to the frontier** with its cost.
 - o If the child is already in the frontier with a **higher cost**, update it with the **lower cost**.

If the new path is better than the old path, then discard the old one.

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	9	c

Unprocessed Nodes = {c, e, f}

Iteration 4



frontier
(priority queue)

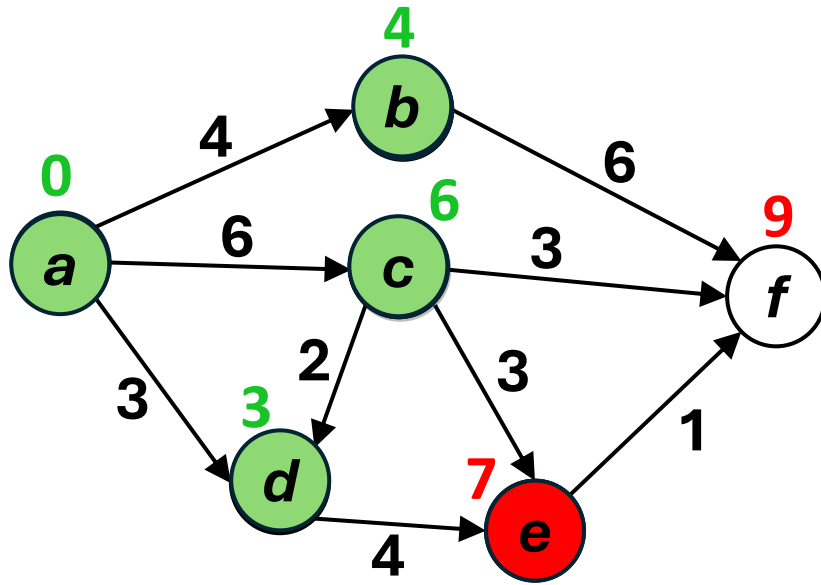
node	dist
e	7
f	9

Processed Nodes = {a, d, b, c}

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	9	c

Unprocessed Nodes = {e, f}

Iteration 5



frontier
(priority queue)

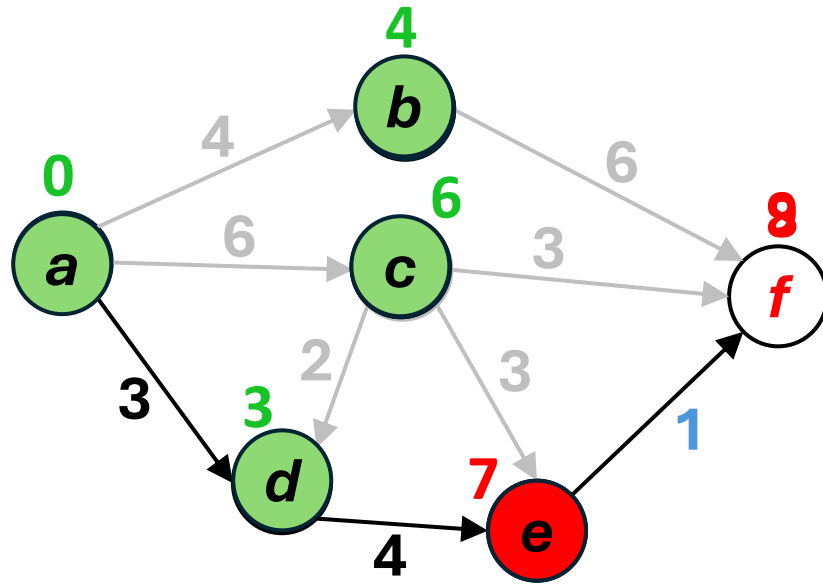
node	dist
e	7
f	9

Processed Nodes = {a, d, b, c}

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	9	c

Unprocessed Nodes = {e, f}

Iteration 5



$$\begin{aligned} \text{dist}[f] &= \text{dist}[e] + 1 \\ &= 7 + 1 \\ &= 8 \end{aligned}$$

Processed Nodes = {a, d, b, c}

frontier
(priority queue)

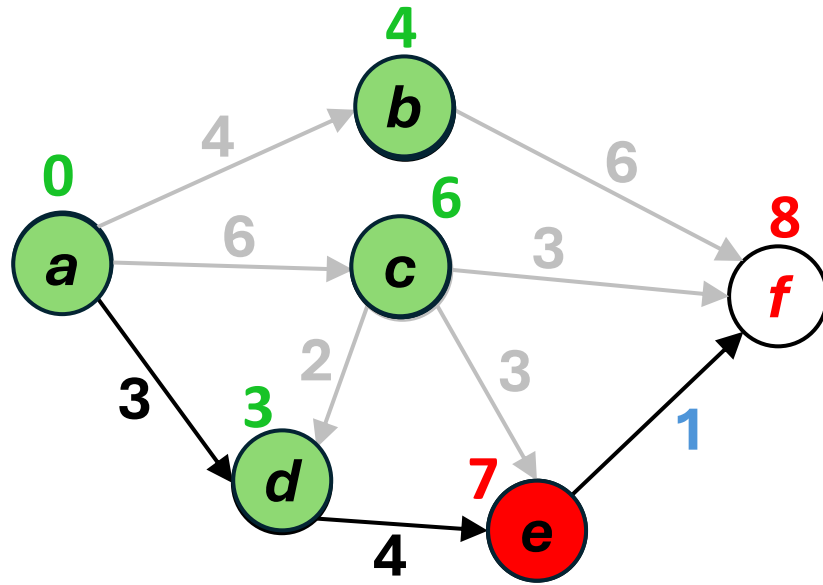
node	dist
f	9

If the new path is better than the old path, then discard the old one.

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	8	e

Unprocessed Nodes = {e, f }

Iteration 5



$$\begin{aligned} \text{dist}[f] &= \text{dist}[e] + 1 \\ &= 7 + 1 \\ &= 8 \end{aligned}$$

Processed Nodes = {*a*, *d*, *b*, *c*}

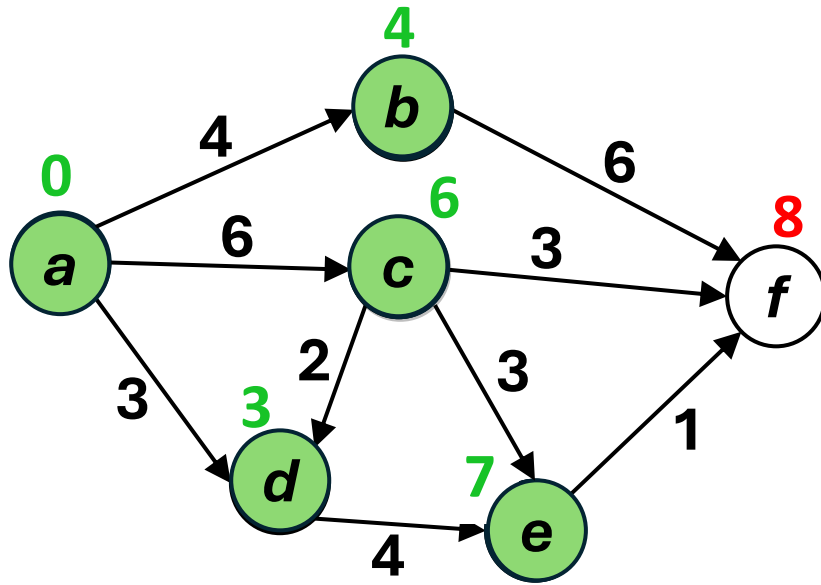
frontier
(priority queue)

node	dist
<i>f</i>	8

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

Unprocessed Nodes = {*e*, *f*}

Iteration 5



frontier
(priority queue)

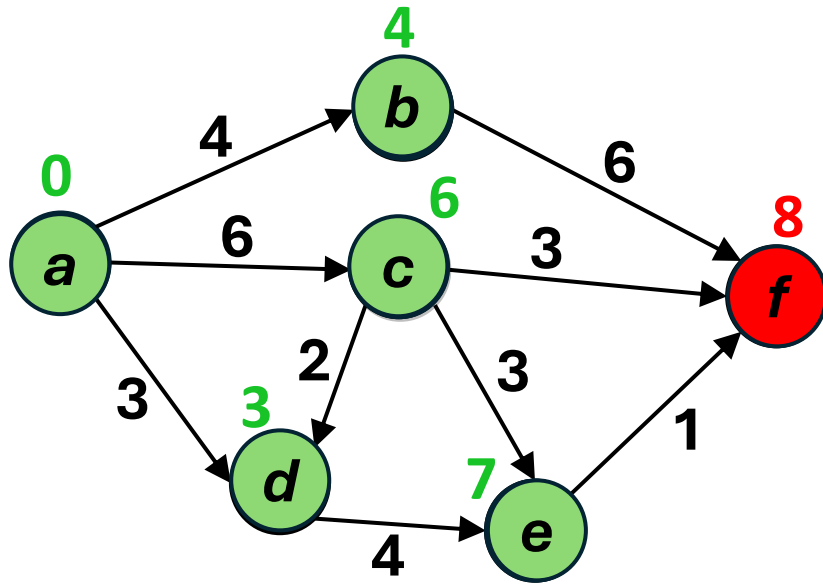
node	dist
<i>f</i>	8

Processed Nodes = {a, d, b, c, e}

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

Unprocessed Nodes = { f }

Iteration 6



frontier
(priority queue)

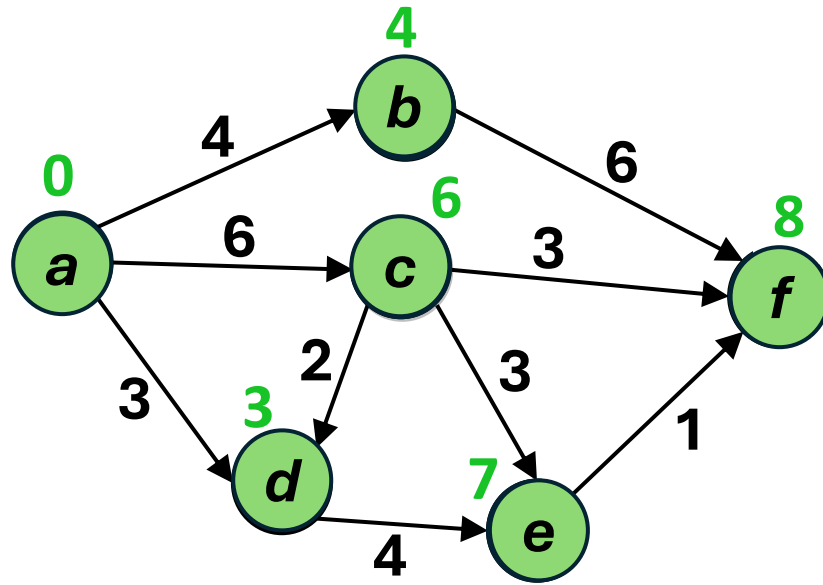
node	dist
<i>f</i>	8

Processed Nodes = {*a*, *d*, *b*, *c*, *e*}

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

Unprocessed Nodes = { *f* }

Iteration 6



frontier
(priority queue)

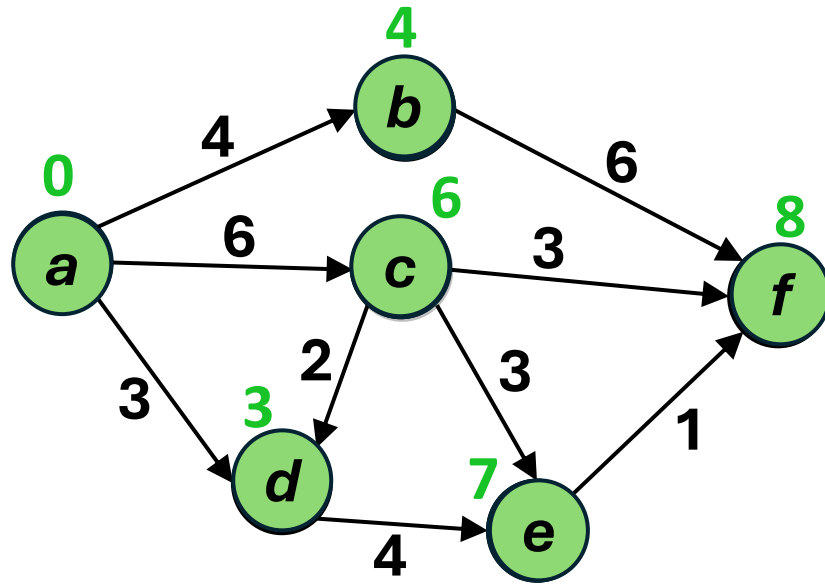
node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

Processed Nodes = {a, d, b, c, e, f}

Unprocessed Nodes = { }

End of procedure



frontier
(priority queue)

node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

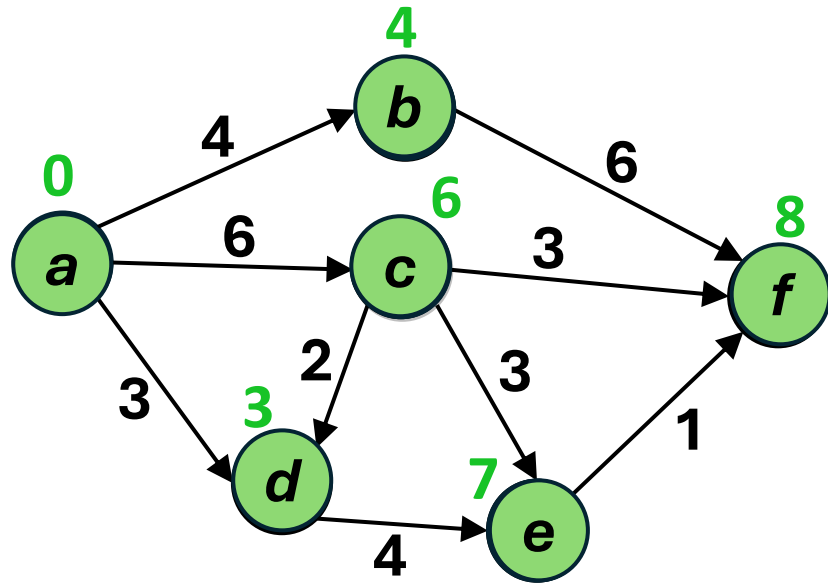
Processed Nodes = {*a*, *d*, *b*, *c*, *e*, *f*}

Unprocessed Nodes = { }

*Since we didn't use a goal test in this example, Dijkstra's algorithm computes the shortest paths from the starting node to all other nodes.

Get the shortest path

Find the shortest path from a to f .

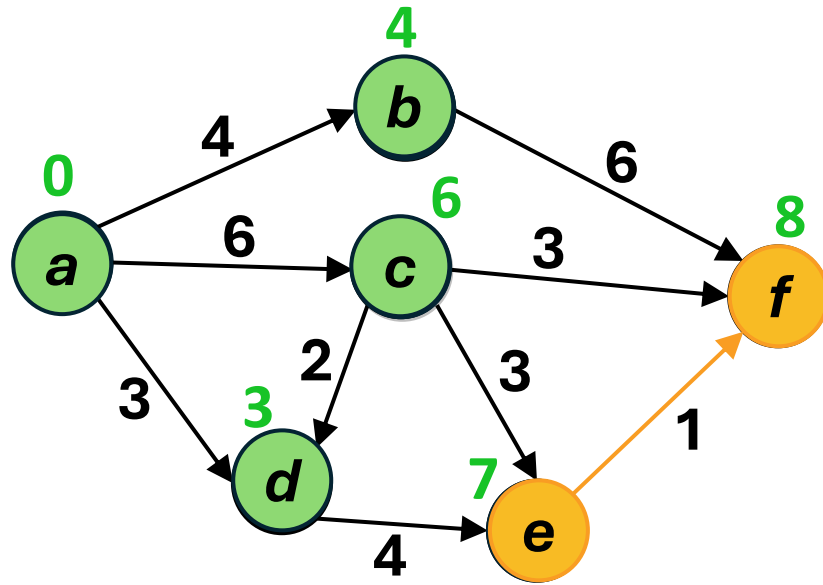


frontier
(priority queue)

node	dist

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	8	e

Get the shortest path



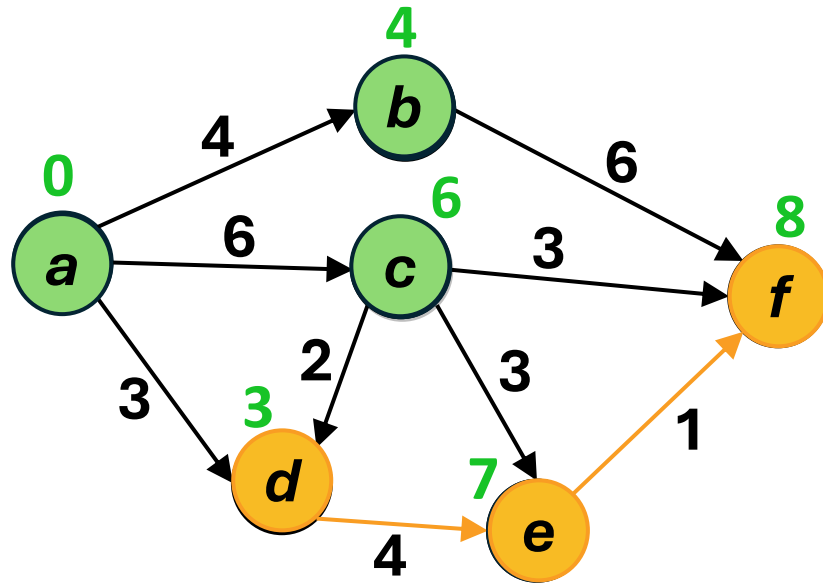
frontier
(priority queue)

node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

f e

Get the shortest path



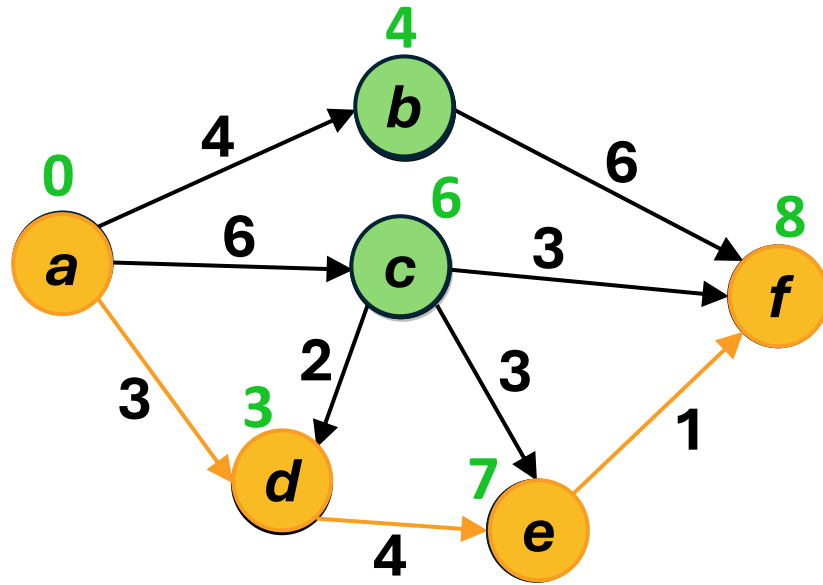
frontier
(priority queue)

node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

f e d

Get the shortest path



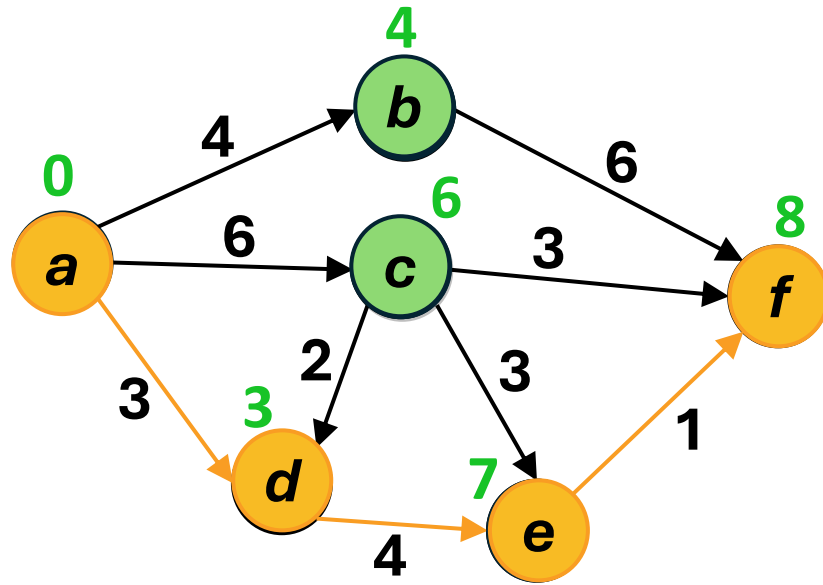
frontier (priority queue)

node	dist
------	------

node	shortest distance from a	previous node
a	0	--
b	4	a
c	6	a
d	3	a
e	7	d
f	8	e

f e d a

Get the shortest path



frontier
(priority queue)

node	dist

node	shortest distance from <i>a</i>	previous node
<i>a</i>	0	--
<i>b</i>	4	<i>a</i>
<i>c</i>	6	<i>a</i>
<i>d</i>	3	<i>a</i>
<i>e</i>	7	<i>d</i>
<i>f</i>	8	<i>e</i>

$path(f) = a \ d \ e \ f$